17. Full Project Documentation

Date	6 July 2026
Team ID	SWTID-2026-8135
Project Name	EduGenie – Google Gemini Powered Learning Assistant
Maximum Marks	(Reference Narrative)

EduGenie: Google Gemini Powered Learning Assistant

Project Description

EduGenie is an AI-powered educational web platform that helps students learn faster by combining Google Gemini's generative reasoning with a local, offline-capable HuggingFace model. The platform bundles five study tools behind one FastAPI backend: a conversational Q&A assistant, a leveled concept explainer, an automatic MCQ quiz generator, a text/notes summarizer, and a personalized learning-roadmap recommender. Every capability is exposed both as a JSON REST API and as a dedicated page in a responsive, glassmorphic-styled web frontend. If no Gemini API key is configured, EduGenie transparently serves realistic demo/mock content instead of failing, so the entire product can be explored end-to-end without any setup cost.

Scenarios

- Scenario 1 — A student preparing for a biology exam opens the Chat page and asks “What is photosynthesis?” EduGenie returns a direct answer, a step-by-step explanation, a real-world example, and 3–4 key takeaway points — generated by Gemini via qna.py and logged to queries.json / responses.json.
- Scenario 2 — A student struggling with “Recursion” opens the Explain page and selects the “simple” level. The local LaMini-Flan-T5 model (loaded lazily at startup) generates a beginner-friendly explanation with analogies; if the local model isn't available, the request transparently falls back to Gemini.
- Scenario 3 — Before a test, a student pastes their “Machine Learning” notes into the Quiz page and requests 5 questions. quiz_module.py prompts Gemini for strictly formatted JSON MCQs, cleans any markdown code fences from the response, and renders an interactive quiz with a timer and scoring in the browser (quiz.js).
- Scenario 4 — A student with a long lecture-notes document pastes it into the Summary page and receives a concise summary, bullet points, and extracted keywords for quick revision.
- Scenario 5 — A self-learner picks “Cybersecurity” on the Recommend page and receives a curated set of resources (book, course, practice platform) plus a multi-step roadmap with estimated study hours per step.

Technical Architecture

EduGenie uses a layered architecture: a Jinja2/HTML/CSS/JS presentation layer, a FastAPI API layer exposing both router-based and direct REST endpoints, a service layer of five feature-specific Python modules, a shared AI-integration layer (utils/gemini_service.py and utils/local_t5_service.py) that unifies retry logic and model resolution, and a JSON-file data layer (utils/db_manager.py) that persists every query and AI response. See Section 10 (Solution Architecture) for the full diagram and component table, and Section 5 (Data Flow Diagram) for how a single request flows through the system.

Prerequisites

- Python 3.10+ and pip
- A Google Gemini API key (optional — the app runs in demo mode without one) from Google AI Studio
- Familiarity with FastAPI, Jinja2 templating, and REST API design
- Docker (optional, for containerized deployment)
- Git for version control

Project Workflow

Milestone 1 — Model Selection and Architecture: Configure the Gemini API key, resolve the best available Gemini model dynamically at startup (utils/gemini_service.py), and define the layered architecture connecting the frontend, FastAPI backend, and AI services.

Milestone 2 — Core Functionalities Development: Implement the five feature modules (qna.py, explanation_module.py, quiz_module.py, summary_module.py, learning_path.py), each with its own Pydantic request/response schema and Gemini/local-model integration with mock-mode fallback.

Milestone 3 — main.py Development: Wire up FastAPI routing — mount static files and templates, register the five feature routers under /api, add direct RESTful endpoints (/qa, /explain/, /quiz, /summarize/, /learn/recommendations), and log every query/response pair to the JSON data store.

Milestone 4 — Frontend Development: Build eight Jinja2 pages (home, chat, explain, quiz, summary, recommend, about, contact) with a shared base layout, glassmorphic CSS styling, a light/dark theme toggle, and per-page JavaScript for dynamic rendering (quiz timers and scoring, copy-to-clipboard, markdown export, accordion explanations).

Milestone 5 — Deployment: Containerize the application with a python:3.10-slim Dockerfile that installs dependencies and runs Uvicorn on port 8000, enabling consistent deployment to any container host.

Milestone 6 — Testing & Conclusion: Validate all routes, request-validation rules and both endpoint styles with an automated pytest suite (tests/test_api.py), then document the finished platform.

Exploring the Website's Web Pages

Home Page (index.html): Landing page introducing EduGenie, with a hero section, feature overview, statistics and FAQ.

Chat Page: Conversational interface for the Q&A module, showing structured answers with explanation, example, and key points.

Explain Page: Lets the student pick a concept and a target level (simple / medium / advanced) and view the generated explanation with examples and analogies.

Quiz Page: Accepts a topic or pasted text, generates MCQs, and runs an interactive quiz experience with a countdown timer, progress tracking and a final score.

Summary Page: Accepts pasted notes and displays a summary, bullet points, and extracted keywords, with a word counter on input.

Recommend Page: Accepts a topic and level, and displays curated resources plus a step-by-step learning roadmap with estimated hours.

About / Contact Pages: Static informational pages describing the project and providing a contact form.

Conclusion

EduGenie demonstrates how a single, well-structured FastAPI application can unify five distinct generative-AI study tools behind a consistent architecture: dynamic Gemini model resolution with retry/backoff, a local offline-capable model for resilience, strict JSON-schema prompting for reliable structured output, and a lightweight JSON-file data layer for full request/response traceability. The project's built-in demo mode makes it fully explorable without any external dependency, which is particularly valuable for academic evaluation and live demonstrations. Looking ahead, the most valuable next steps are consolidating the duplicate API endpoint surface, migrating from JSON files to a proper relational database for concurrent-write safety, and adding user authentication so learning history and quiz performance can be tracked per student over time.